COMP0222 Simultaneous Localisation and Mapping

Lecture 07: Inference and Optimization on Factor Graphs

Simon Julier

s.julier@ucl.ac.uk

Sajad Saeedi

s.saeedi@ucl.ac.uk

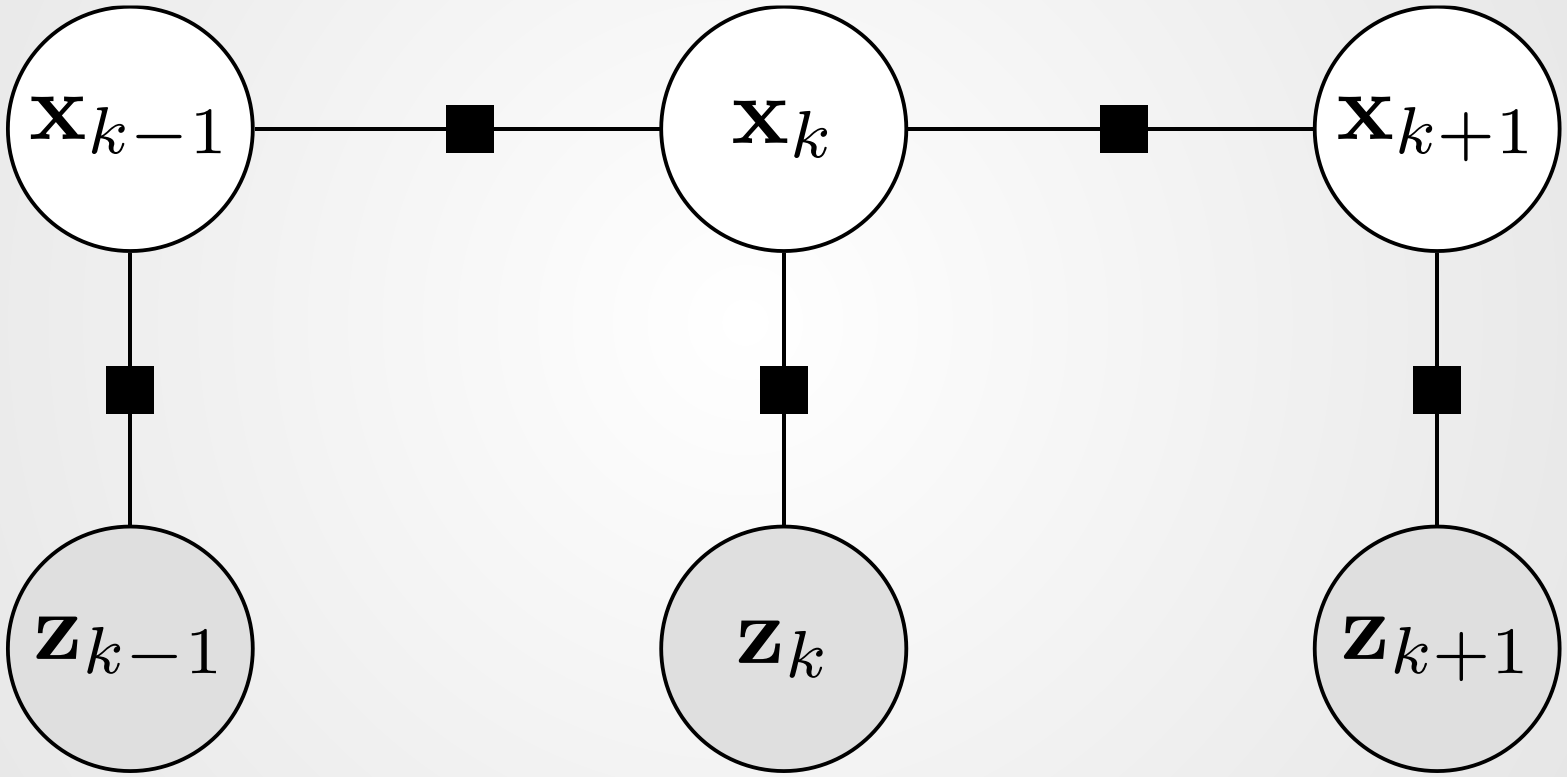
Structure

- The Problem with Expected Values
- Maximum A Posteriori Estimation
- Optimization Algorithms
- Approximate Covariance Calculations
- Common Optimization Algorithms

The Problem with Expected Values

- **The Problem with Expected Values**
- ***Maximum A Posteriori Estimation***
- ***Optimization Algorithms***
- ***Approximate Covariance Calculations***
- ***Common Optimization Algorithsm***

Motivation


$\mathbf{x}_{k-1}$
$\mathbf{x}_k$
$\mathbf{x}_{k+1}$
$\mathbf{z}_{k-1}$
$\mathbf{z}_k$
$\mathbf{z}_{k+1}$

Computing the Mean

- Suppose we want to compute the mean of the *j*th state
- The mean is given by

$$\mathbb{E}\left[\mathbf{x}_j\right] = \int \mathbf{x}_j f\left(\mathbf{x}_j; \mathbf{z}_{1:k}, \mathbf{u}_{1:k}\right) \mathrm{d}\mathbf{x}_j$$

Marginalization

- The first thing is we need is

$$f\left(\mathbf{x}_j; \mathbf{z}_{1:k}, \mathbf{u}_{1:k}\right)$$

- However, the factor graph gives us a PDF proportional to

$$f\left(\mathbf{x}_{0:k} \middle| \mathbf{z}_{1:k}; \mathbf{u}_{1:k}\right)$$

Marginalization

- **We get rid of the other variables through *marginalization*,**

$$f\left(\mathbf{x}_j|\mathbb{I}_k\right) = \int f\left(\mathbf{x}_{0:k}|\mathbb{I}_k\right)\mathrm{d}\mathbf{x}_{0:k\backslash j}$$

$$0:k\backslash j = \{0, 1, 2, \cdots, j-1, j+1, \cdots, k\}$$

Marginalization

- However, performing marginalization is a *nightmare*
- Even then, we aren't done
- We still have to compute the mean using the marginal distribution
- There are various numerical techniques (e.g., sampling) which will generate approximate solutions
- However, these scale badly with dimensions and SLAM problems get very big

Working with Estimates other than the Mean

- We want to get an estimate from our PDF *without having to integrate anything*
- It turns out there is an estimation technique called Maximum A Posterior (MAP) estimation
- This turns estimation into a peak finding problem

Maximum A Posteriori Estimation

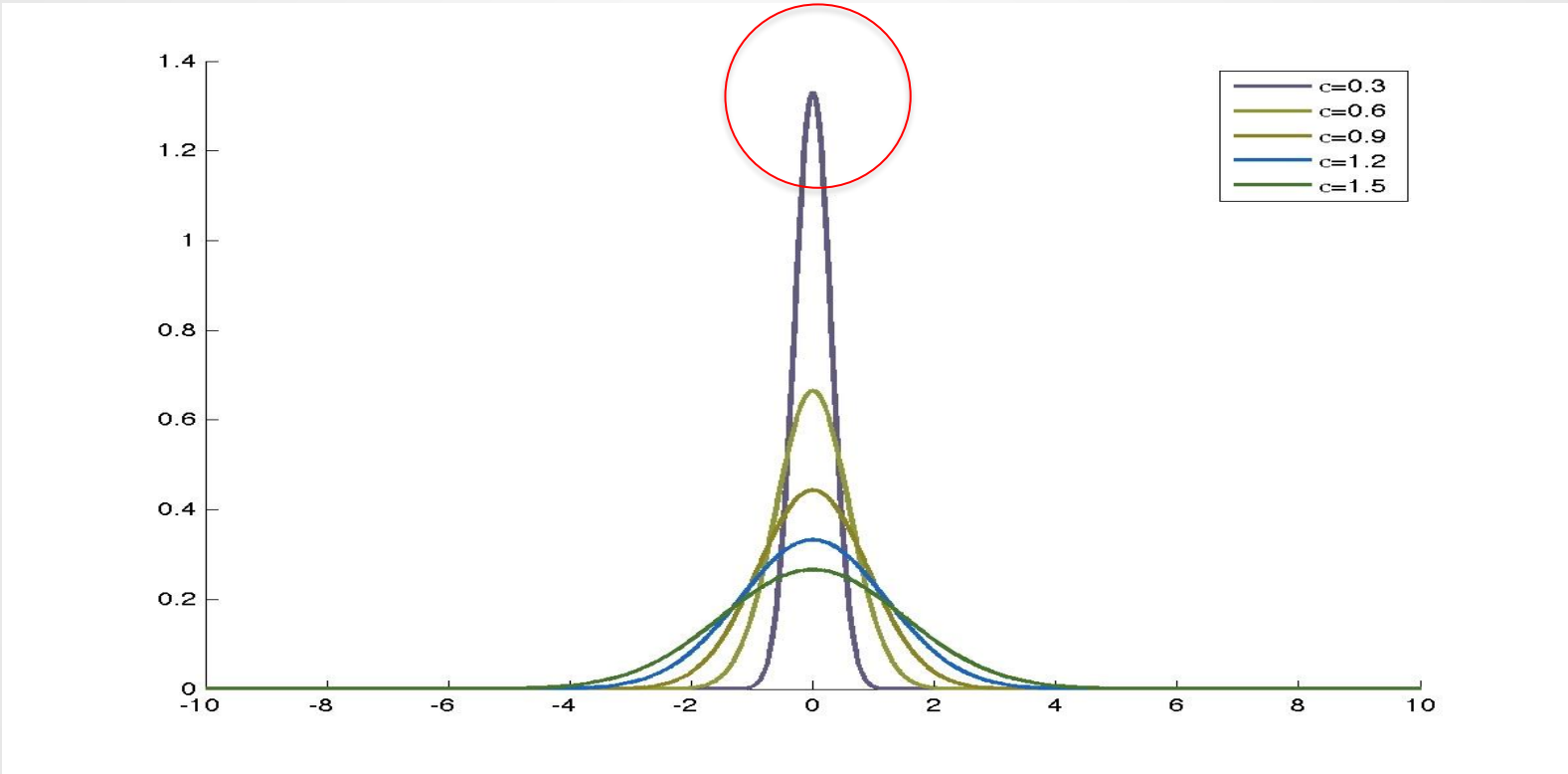
- *The Problem with Expected Values*
- **Maximum A Posteriori Estimation**
- *Optimization Algorithms*
- *Approximate Covariance Calculations*
- *Common Optimization Algorithms*

Maximum A Posteriori Estimation

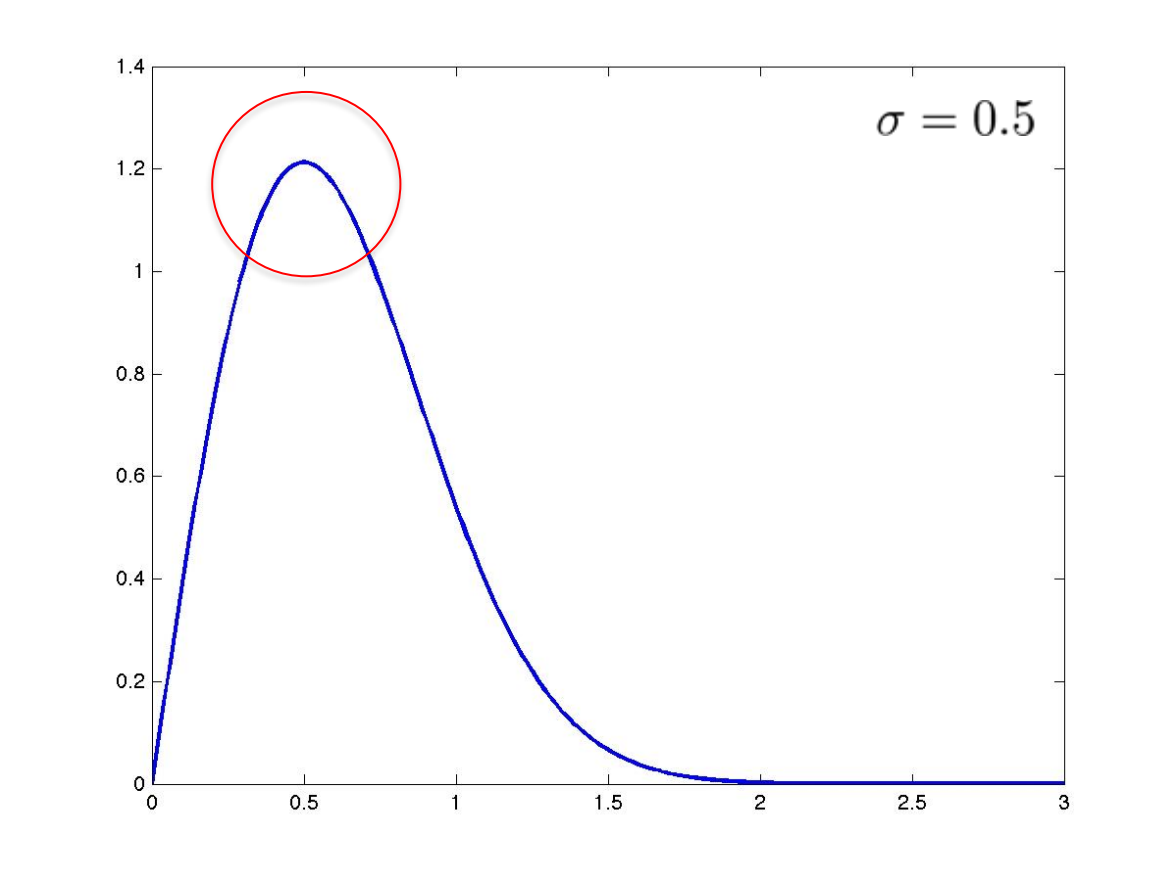
- **The output estimate is the state value which *maximises the numerical value* of the PDF**

$$\hat{\mathbf{x}}_{0:k} = \arg\max_{\mathbf{x}_{0:k}} f\left(\mathbf{x}_{0:k} \middle| \mathbb{I}_k\right)$$

MAP of a Gaussian PDF



MAP of a Rayleigh PDF



Maximum A Posteriori Estimation

- The output estimate is the state value which maximises the PDF

$$\hat{\mathbf{x}}_{0:k} = \arg\max_{\mathbf{x}_{0:k}} f\left(\mathbf{x}_{0:k}; \mathbf{u}_{1:k}, \mathbf{v}_{1:k}\right)$$

- Advantages:
  - No integration at all
  - The estimate itself is often pretty sensible
  - It's often fast to compute

Normalization Constant not Required [REDRAW c > 0]

- Suppose we want to solve

$$\hat{\mathbf{x}}_{0:k} = \arg\max_{\mathbf{x}_{0:k}} f\left(\mathbf{x}_{0:k}; \mathbf{u}_{1:k}, \mathbf{v}_{1:k}\right)$$

- We get the *same* solution from the equation,

$$\hat{\mathbf{x}}_{0:k} = \arg\max_{\mathbf{x}_{0:k}} c f\left(\mathbf{x}_{0:k}; \mathbf{u}_{1:k}, \mathbf{v}_{1:k}\right)$$

$$c > 0$$

Computing the MAP Estimate

- **From the factor graph, we know that the form of the PDF is**

$$f\left(\mathbf{x}_{0:k}|\mathbf{z}_{1:k};\mathbf{u}_{1:k}\right) \propto f\left(\mathbf{x}_0\right)\prod_{i=1}^{k} f\left(\mathbf{x}_i|\mathbf{x}_{i-1};\mathbf{u}_i\right)$$

$$\times \prod_{i=1}^{k} L\left(\mathbf{x}_i;\mathbf{z}_i\right)$$

Computing the MAP Estimate

- The joint PDF is a product of lots of terms or factors
- Since these are PDFs and unnormalized, so the only thing we know about the factors is that they are non-negative
- Therefore, multiplying lots of factors together can lead to numerical overflow and underflow issue
- We exploit the product of non-negative terms and take the *log* of the distribution

MAP and Logs

- Since log is an asymptotically increasing function, the *location* of the maximum will not change,

$$\hat{\mathbf{x}}_{0:k} = \arg\max_{\mathbf{x}_{0:k}} f\left(\mathbf{x}_{0:k}; \mathbf{u}_{1:k}, \mathbf{v}_{1:k}\right)$$

$$= \arg\max_{\mathbf{x}_{0:k}} \ln f\left(\mathbf{x}_{0:k}; \mathbf{u}_{1:k}, \mathbf{v}_{1:k}\right)$$

Computing the MAP Estimate

- **Substituting,**

$$\ln f\left(\mathbf{x}_{0:k}|\mathbf{z}_{1:k};\mathbf{u}_{1:k}\right) = \ln f\left(\mathbf{x}_0\right) + \sum_{i=1}^{k} \ln f\left(\mathbf{x}_i|\mathbf{x}_{i-1};\mathbf{u}_i\right)$$

$$+ \sum_{i=1}^{k} \ln L\left(\mathbf{x}_i;\mathbf{z}_i\right)$$

Functional Forms of the Different Factors

- To proceed further, we need to write down in a concrete manner what the factors look like
- We are going to assume that the initial conditions, process and observation noises are all Gaussian and zero-mean distributed
- There are extensions to the factor graph which avoid this limitation, which we probably won't cover in this module

Initial Condition

- We first have to specify the PDF for the initial conditions
- Suppose the initial condition is Gaussian distributed,

$$\mathbf{x_0} \sim \mathcal{G}\left(\mathbf{x_0}; \bar{\mathbf{x}}_\mathbf{0}, \mathbf{P}_0\right)$$

PDF of the Initial Condition

- **Therefore,**

$$f\left(\mathbf{x_0}\right) \propto \exp\left\{-\frac{1}{2}\left(\mathbf{x_0}-\bar{\mathbf{x}}_\mathbf{0}\right)^\top \mathbf{P_0^{-1}}\left(\mathbf{x_0}-\bar{\mathbf{x}}_\mathbf{0}\right)\right\}$$

- **Taking logs,**

$$\ln f\left(\mathbf{x_0}\right) = -\frac{1}{2}\left(\mathbf{x_0}-\bar{\mathbf{x}}_\mathbf{0}\right)^\top \mathbf{P_0^{-1}}\left(\mathbf{x_0}-\bar{\mathbf{x}}_\mathbf{0}\right) + \boldsymbol{A_0}$$

Dropping the Normalizing Constant

- Since it doesn't make any difference to the MAP estimation problem, we'll drop the normalizing constant
- Therefore, we'll use the expression

$$\ln f\left(\mathbf{x}_0\right) = -\frac{1}{2}\left(\mathbf{x}_0 - \bar{\mathbf{x}}_0\right)^\top \mathbf{P}_0^{-1}\left(\mathbf{x}_0 - \bar{\mathbf{x}}_0\right)$$

State Transition Probabilities and Likelihoods

- **We now need to substitute for the expressions for the transition density and likelihood functions**

$$f\left(\mathbf{x}_{k+1}|\mathbf{x}_k;\mathbf{u}_{k+1}\right)=f_{\mathbf{v}}\left(\mathbf{v}_{k+1}=\mathbf{e}\left[\mathbf{x}_{k+1},\mathbf{x}_k,\mathbf{u}_{k+1}\right]\right)$$
$$L\left(\mathbf{x}_{k+1};\mathbf{z}_{k+1}\right)=f_{\mathbf{w}}\left(\mathbf{w}_{k+1}=\mathbf{l}\left[\mathbf{z}_{k+1},\mathbf{x}_{k+1}\right]\right)$$

State Transition Probability

- **Given the assumption that the process noise is a zero mean Gaussian with covariance $\mathbf{Q}_k$, the PDF has the form**

$$f\left(\mathbf{x}_{k+1}|\mathbf{x}_k;\mathbf{u}_{k+1}\right) \propto \exp\left\{-\frac{1}{2}\mathbf{e}\left[\mathbf{x}_{k+1},\mathbf{x}_k,\mathbf{u}_{k+1}\right]^\top \mathbf{Q}_k^{-1}\mathbf{e}\left[\mathbf{x}_{k+1},\mathbf{x}_k,\mathbf{u}_{k+1}\right]\right\}$$

- **Taking logs,**

$$\ln f\left(\mathbf{x}_{k+1}|\mathbf{x}_k;\mathbf{u}_{k+1}\right) = -\frac{1}{2}\mathbf{e}\left[\mathbf{x}_{k+1},\mathbf{x}_k,\mathbf{u}_{k+1}\right]^\top \mathbf{Q}_k^{-1}\mathbf{e}\left[\mathbf{x}_{k+1},\mathbf{x}_k,\mathbf{u}_{k+1}\right]$$

Likelihood Function

- **Similarly, for the likelihood equations,**

$$L\left(\mathbf{x}_{k+1};\mathbf{z}_{k+1}\right) \propto \exp\left\{-\frac{1}{2}\mathbf{l}\left[\mathbf{z}_{k+1},\mathbf{x}_{k+1}\right]^{\top}\mathbf{R}_{k+1}^{-1}\mathbf{l}\left[\mathbf{z}_{k+1},\mathbf{x}_{k+1}\right]\right\}$$

- **Taking logs,**

$$\ln L\left(\mathbf{x}_{k+1};\mathbf{z}_{k+1}\right) = -\frac{1}{2}\mathbf{l}\left[\mathbf{z}_{k+1},\mathbf{x}_{k+1}\right]^{\top}\mathbf{R}_{k+1}^{-1}\mathbf{l}\left[\mathbf{z}_{k+1},\mathbf{x}_{k+1}\right]$$

Putting Everything Together

Adding everything together,

$$
\begin{aligned}
\ln f\left(\mathbf{x}_{0:k} \middle| \mathbf{z}_{1:k}; \mathbf{u}_{1:k}\right) = & -\frac{1}{2}\left(\mathbf{x}_0 - \bar{\mathbf{x}}_0\right)^\top \mathbf{P}_0^{-1}\left(\mathbf{x}_0 - \bar{\mathbf{x}}_0\right) \\
& - \frac{1}{2}\sum_{i=1}^{k} \mathbf{e}\left[\mathbf{x}_i, \mathbf{x}_{i-1}, \mathbf{u}_i\right]^\top \mathbf{Q}_i^{-1} \mathbf{e}\left[\mathbf{x}_i, \mathbf{x}_{i-1}, \mathbf{u}_i\right] \\
& - \frac{1}{2}\sum_{i=1}^{k} \mathbf{l}\left[\mathbf{z}_i, \mathbf{x}_i\right]^\top \mathbf{R}_i^{-1} \mathbf{l}\left[\mathbf{z}_i, \mathbf{x}_i\right]
\end{aligned}
$$

Putting Everything Together

- Our MAP estimate is the solution to

$$\hat{\mathbf{x}}_{0:k} = \arg\max_{\mathbf{x}_{0:k}} \ln f\left(\mathbf{x}_{0:k}; \mathbf{u}_{1:k}, \mathbf{v}_{1:k}\right)$$

- However, the cost function has lots of -1/2 in them which is bit inconvenient
- Also most optimizers try to work out the *minimum* value of a function instead of its *maximum*

Putting Everything Together

- Therefore, we define the cost function to be

$$c\left[\mathbf{x}_{0:k};\mathbf{u}_{1:k},\mathbf{v}_{1:k}\right]=-2\ln f\left(\mathbf{x}_{0:k};\mathbf{u}_{1:k},\mathbf{v}_{1:k}\right)$$

- The MAP estimate is now obtained by solving the optimization problem,

$$\hat{\mathbf{x}}_{0:k}=\arg\min_{\mathbf{x}_{0:k}} c\left[\mathbf{x}_{0:k};\mathbf{u}_{1:k},\mathbf{v}_{1:k}\right]$$

Putting Everything Together

- **Substituting,**

$$
\begin{aligned}
c\left[\mathbf{x}_{0:k}; \mathbf{u}_{1:k}, \mathbf{v}_{1:k}\right] = & \left(\mathbf{x}_0 - \bar{\mathbf{x}}_0\right)^\top \mathbf{P}_0^{-1} \left(\mathbf{x}_0 - \bar{\mathbf{x}}_0\right) \\
& + \sum_{i=1}^{k} \mathbf{e}\left[\mathbf{x}_i, \mathbf{x}_{i-1}, \mathbf{u}_i\right]^\top \mathbf{Q}_i^{-1} \mathbf{e}\left[\mathbf{x}_i, \mathbf{x}_{i-1}, \mathbf{u}_i\right] \\
& + \sum_{i=1}^{k} \mathbf{l}\left[\mathbf{z}_i, \mathbf{x}_i\right]^\top \mathbf{R}_i^{-1} \mathbf{l}\left[\mathbf{z}_i, \mathbf{x}_i\right]
\end{aligned}
$$

Nonlinear Least Squares Optimization

- **Finding the MAP estimate, assuming the noises are Gaussian, is a nonlinear least squares optimization problem**
- **Nonlinear systems rarely have a closed form solution**
- **Therefore, we need to use *a nonlinear iterative optimizer* to find the solution**

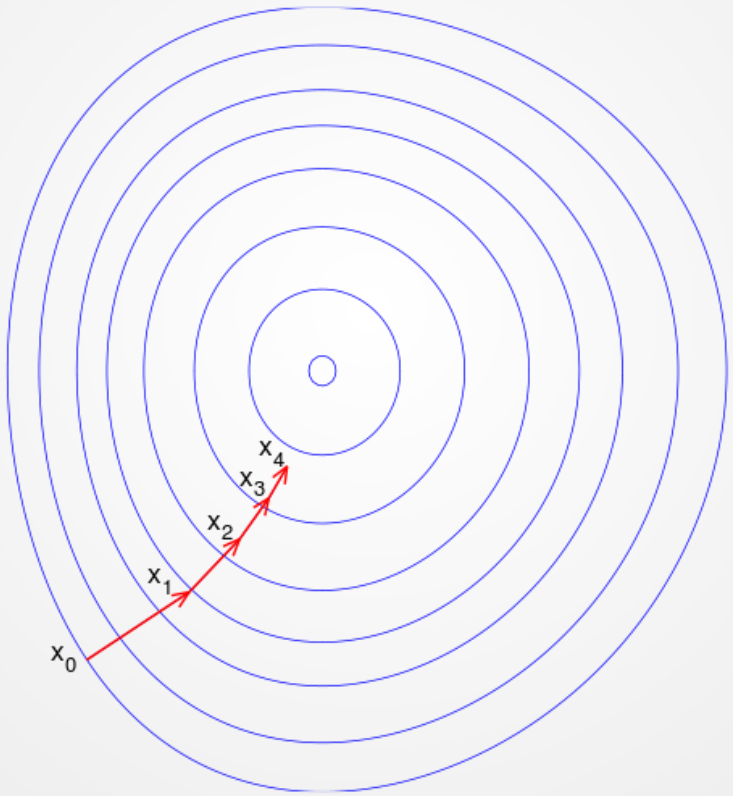
Optimization Algorithms

- *The Problem with Expected Values*
- *Maximum A Posteriori Estimation*
- Optimization Algorithms
- *Approximate Covariance Calculations*
- *Common Optimization Algorithms*

Nonlinear Least Squares Optimization

- This is an incredibly common across maths, engineering, physics machine learning and robotics
- Fortunately, there are a huge number of highly optimized algorithms which have been created over decades
- Almost all of these are *iterative*
- You generate a sequence of candidate solutions which hopefully get better and better

Iterative Nonlinear Optimization



$$\mathbf{X}_k^i$$

Generic Nonlinear Optimization Algorithm

$$
\begin{aligned}
&\mathbf{X}_k^0 \leftarrow \text{Initial value} \\
&c^0 \leftarrow c\left(\mathbf{X}_k^0\right) \\
&i \leftarrow 1 \\
&\textbf{repeat} \\
&\quad \mathbf{v}_k^i \leftarrow \text{Step direction} \\
&\quad \mu_k^i \leftarrow \text{Step length} \\
&\quad \mathbf{X}_k^i = \mathbf{X}_k^{i-1} + \mu_k^i \mathbf{v}_k^i \\
&\quad c^i \leftarrow c\left(\mathbf{X}_k^i\right) \\
&\quad i \leftarrow i + 1 \\
&\textbf{until } \left|c^i - c^{i-1}\right| \leq \epsilon_c \text{ or } \left|\mathbf{X}_k^i - \mathbf{X}_k^{i-1}\right| \leq \epsilon_X
\end{aligned}
$$

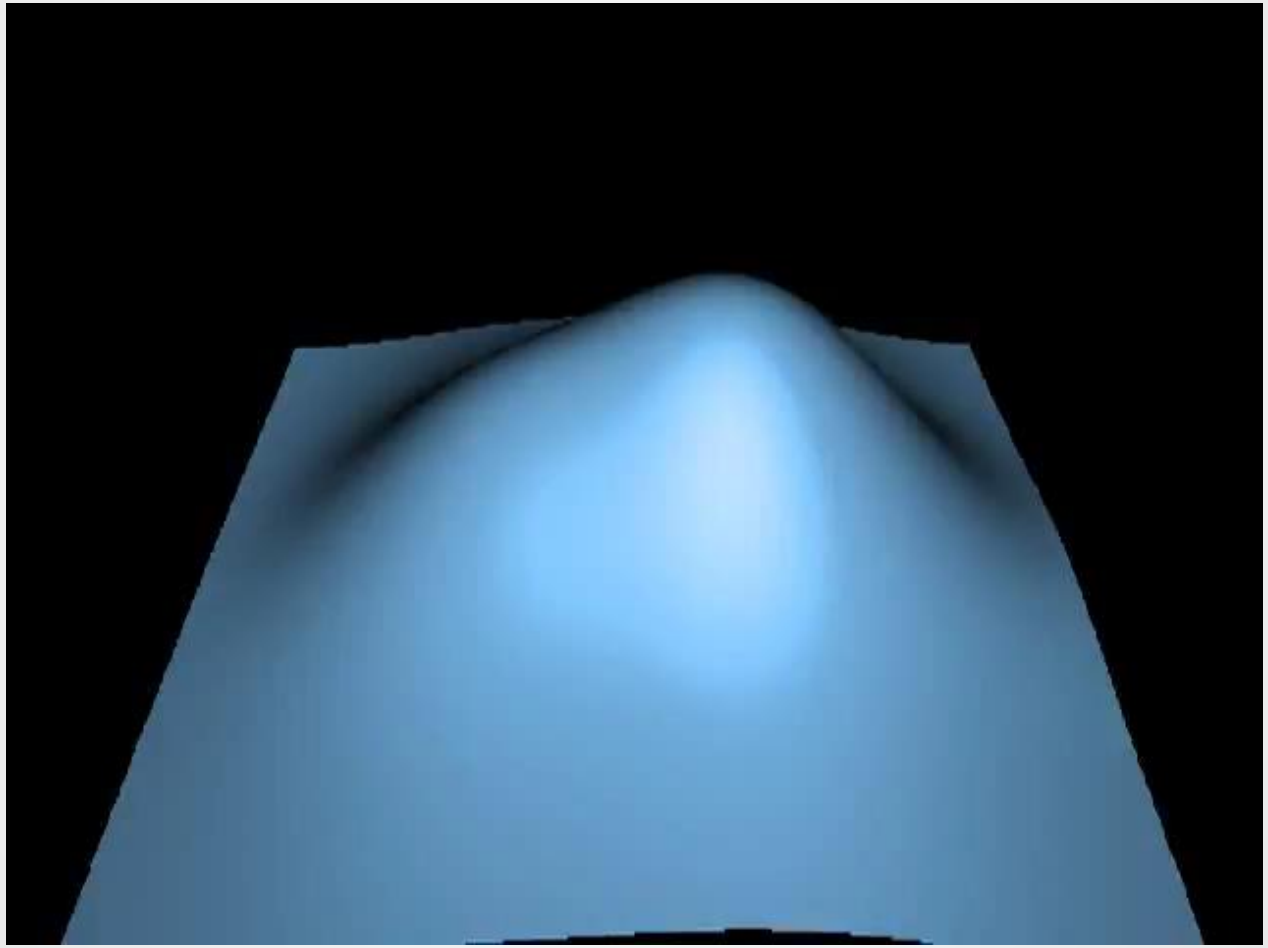
Gradient Descent Optimization

- **Chooses the direction in which the cost function will fall the fastest**
- **The direction of greatest descent is the *negative* of the gradient of the cost function**

$$\mathbf{X}_k^{i+1} = \mathbf{X}_k^i - \mathbf{J}\left(\mathbf{X}_k^i\right)$$

$$\mathbf{J} = \boldsymbol{\nabla} c\left(\mathbf{X}_k^i\right) = \left[\frac{\partial c\left(\mathbf{X}_k^i\right)}{\partial x_1} \quad \ldots \quad \frac{\partial c\left(\mathbf{X}_k^i\right)}{\partial x_n}\right]^\top$$

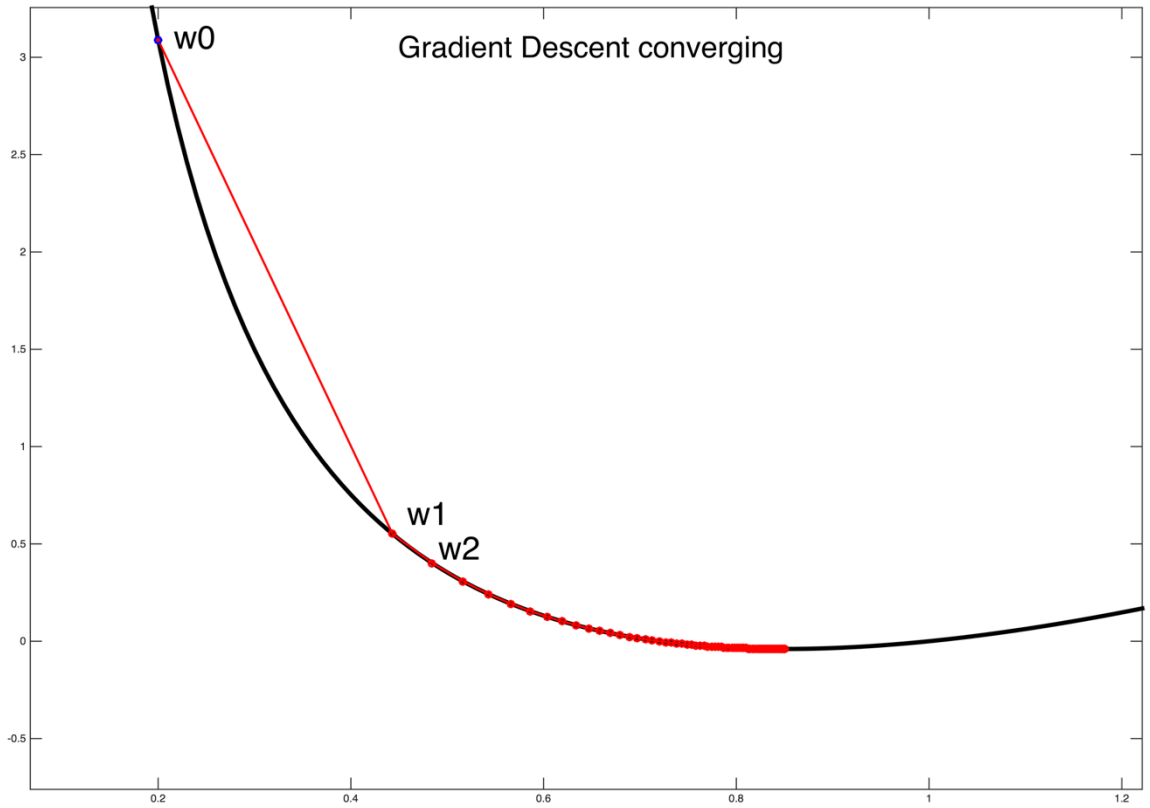
Gradient Descent (Ascent!) at Work



From https://www.youtube.com/watch?v=L2W5SfGu09M

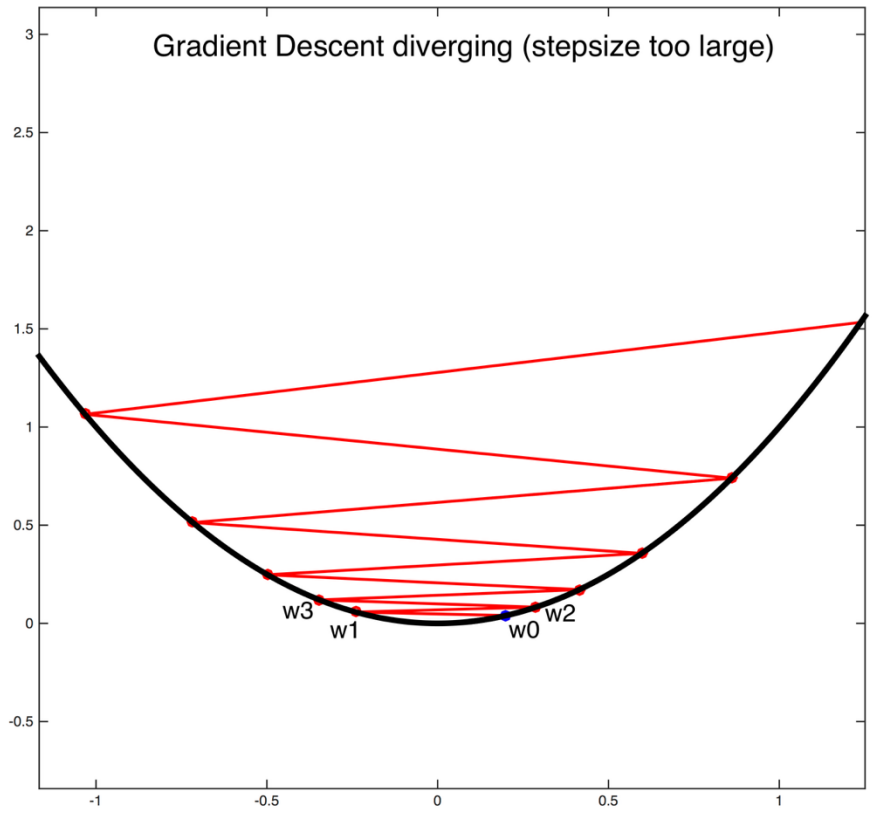
Effect of the Update

Good Step Size



From "Gradient Descent (and Beyond)"

Bad Step Size



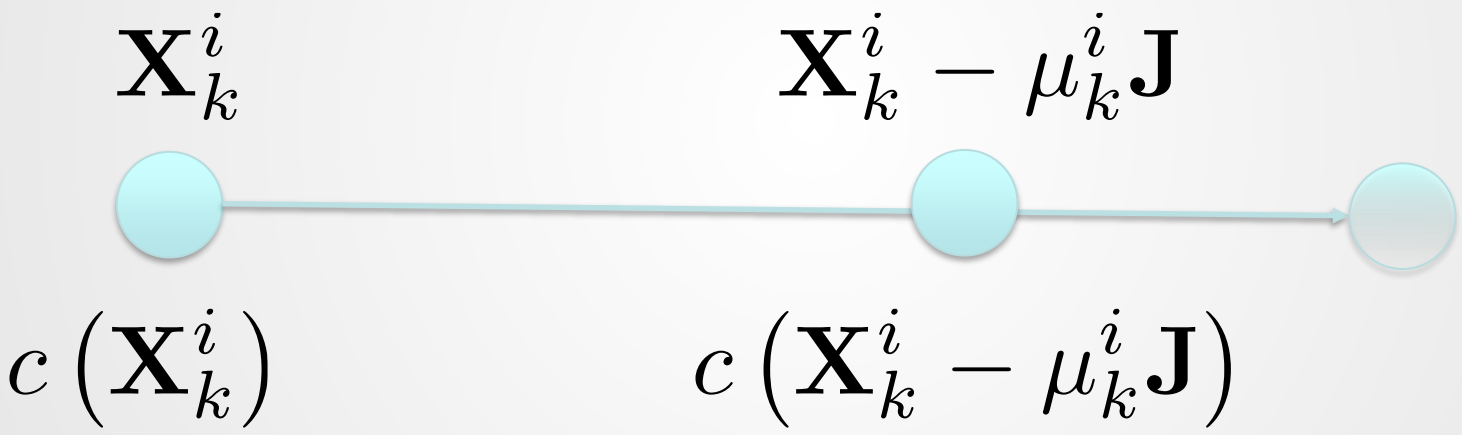
From "Gradient Descent (and Beyond)"

Scaling the Step Size

- We could try just changing the optimization direction to find somewhere "better"
- However most approaches assume that we simply made our step size "too big"
- Therefore, the algorithm takes the *scaled* step

$$\mathbf{X}_k^{i+1} = \mathbf{X}_k^i - \mu_k^i \mathbf{J}\left(\mathbf{X}_k^i\right) \quad 0 < \mu_k^i \leq 1$$

Scaling the Step Size



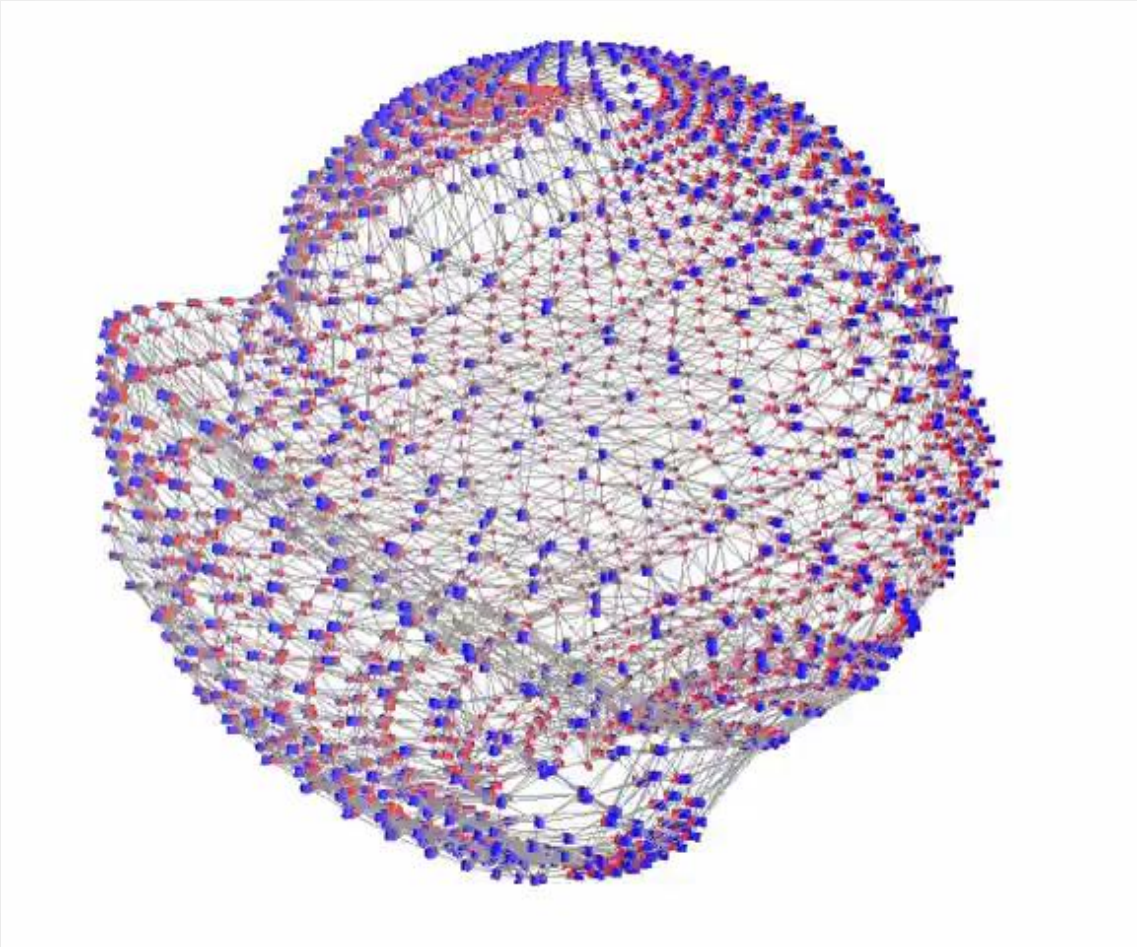
Choosing the Step Size

- **Most versions use some kind of *backtracking line search***
- **Try a jump; if it's bad try a smaller size**

Choosing the Step Size

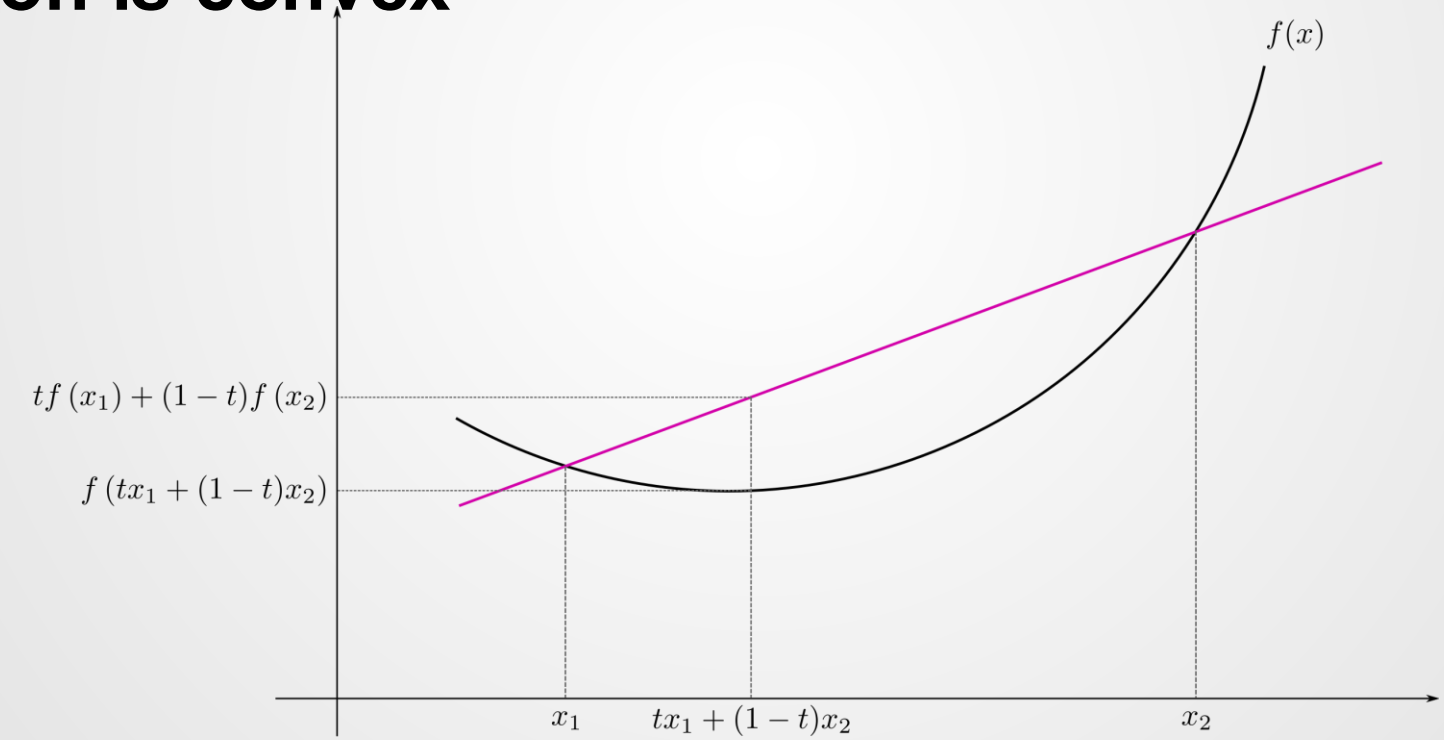
- **In the next step, scale the step size up by a factor β and do the search again**

Challenge Optimization Problem



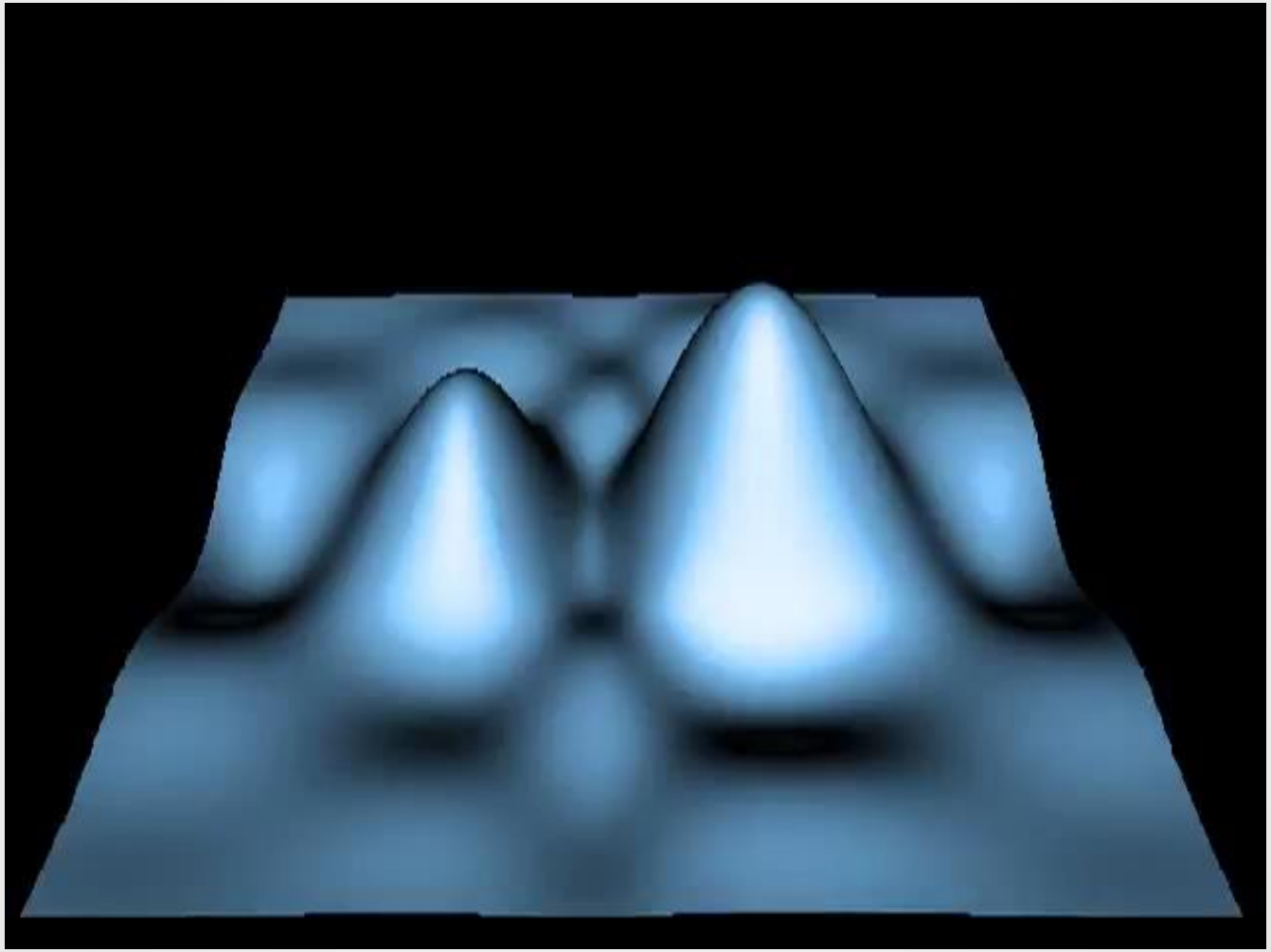
Convexity and Optimization

- **Gradient descent approaches are guaranteed to work only if the function is convex**



From https://en.wikipedia.org/wiki/Convex_function

Gradient Ascent and non-Convex Functions



Addressing Issues of Non-Convexity

- Almost any real system is non-convex
- However, they have a “basin of attraction” for the solution
- The rule-of-thumb is to try to compute an initial solution which *is as close as possible to the optimum*
- There are new “certifiable” SLAM algorithms which guarantee optimal performance over slightly more general conditions

How Accurate is My Estimate?

- We can use nonlinear optimization algorithms to find the MAP estimate
- However, we are often interested in *how accurate* the estimate is if, for example, we have risk aware control
- However, factor graphs make it possible to compute an approximate solution which is good enough in many cases

Approximate Covariance Calculations

- *The Problem with Expected Values*
- *Maximum A Posteriori Estimation*
- *Optimization Algorithms*
- **Approximate Covariance Calculations**
- *Common Optimization Algorithms*

Computing the Covariance

- The covariance of a state estimate is *never used* when computing the MAP estimate
- Computing it properly is infeasible – we'd have to figure out that marginal distribution first and then we still would have to compute the mean
- Fortunately, we can get a rough estimate from the *Laplace Approximation*

Laplace Approximation

- Consider an arbitrary nonlinear distribution
- We approximate it about the MAP by a Gaussian
- The mean of the Gaussian is the MAP estimate
- The covariance of the Gaussian is given by the local curvature of the distribution
- This is a function of the derivative of the cost function, and requires no integration

Laplace Approximation

- **Define the function which is the *log* of the PDF provided by the factor graph,**

$$g(x) = \ln f(x)$$

Laplace Approximation

- **We can write this in terms of its Taylor Series expansion about the MAP estimate,**

$$g(x) = g(\hat{x}^*) + g'(\hat{x}^*)(x - \hat{x}^*) + \frac{1}{2}g''(\hat{x}^*)(x - \hat{x}^*)^2 + \cdots$$

Laplace Approximation

- Since the MAP estimate is a local maxima, it must obey two conditions
- The first derivative must be zero

$$g'(\hat{x}^*) = 0$$

- The second derivative must be negative

$$g''(\hat{x}^*) < 0$$

Laplace Approximation

- **Therefore, the Taylor series expansion about the MAP estimate is**

$$g(x) = g(\hat{x}^*) + \frac{1}{2} g''(\hat{x}^*)(x - \hat{x}^*)^2 + \cdots$$

Laplace Approximation

- Suppose we approximate this by a Gaussian, whose mean is the same as the MAP estimate
- The negative log Taylor series expansion is given by

$$\tilde{g}(x) = -\frac{(x - \hat{x}^*)^2}{2\sigma^2}$$

Laplace Approximation

- **Comparing the two series**

$$g(x) = g(\hat{x}^*) + \frac{1}{2} g''(\hat{x}^*)(x - \hat{x}^*)^2 + \cdots$$

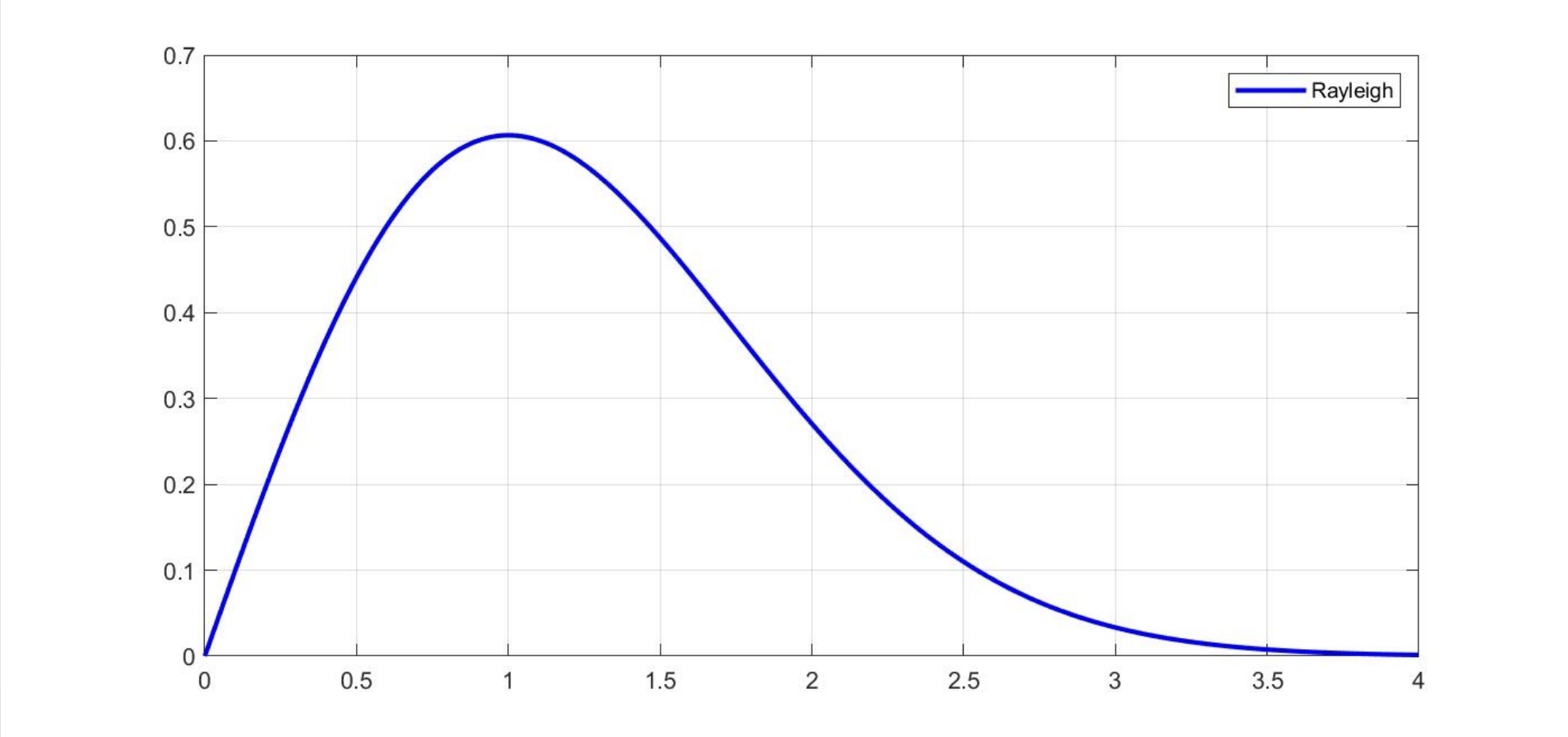
$$\tilde{g}(x) = -\frac{(x - \hat{x}^*)^2}{2\sigma^2}$$

Laplace Approximation

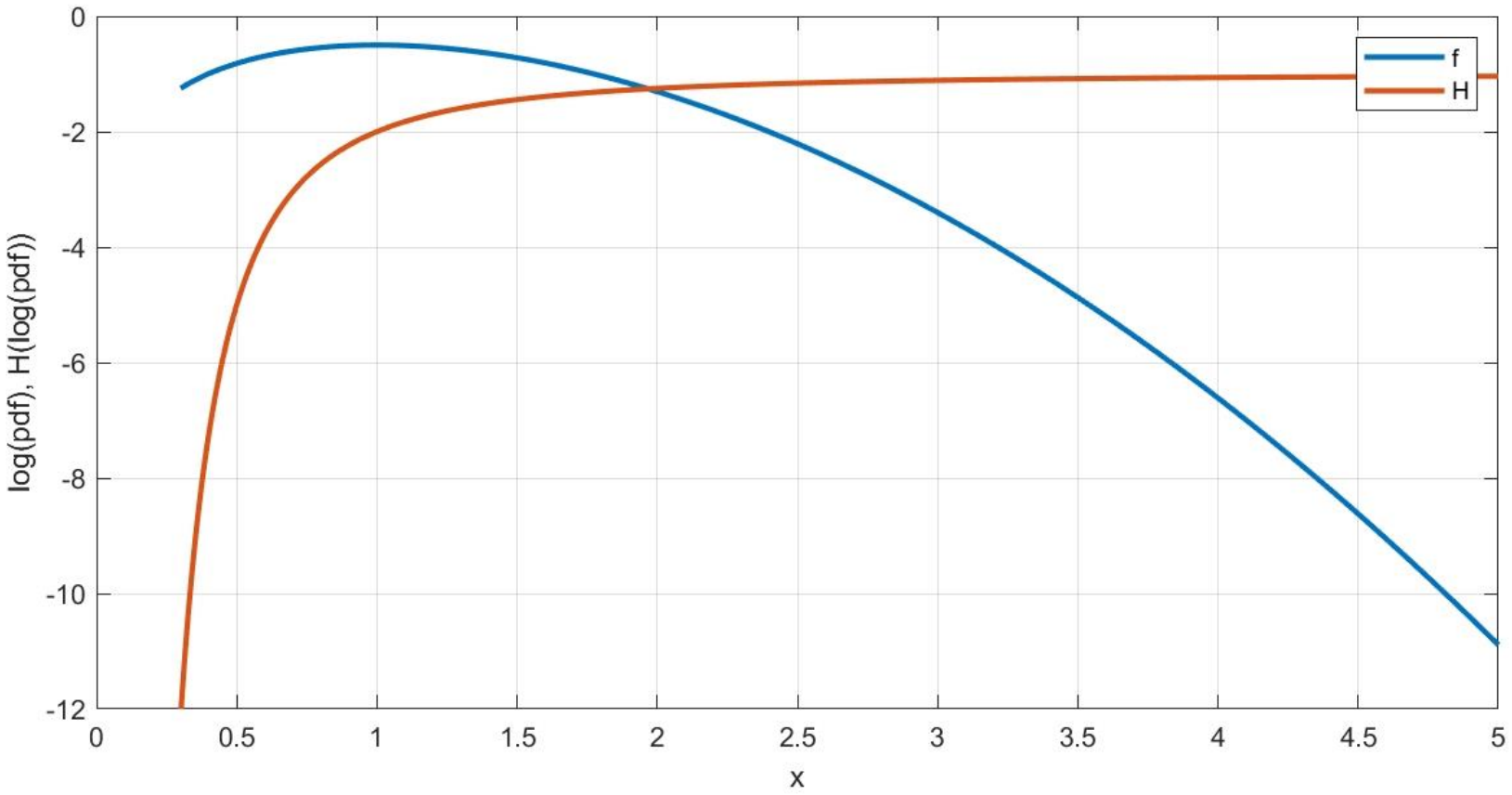
- **Therefore, the covariance is approximated as**

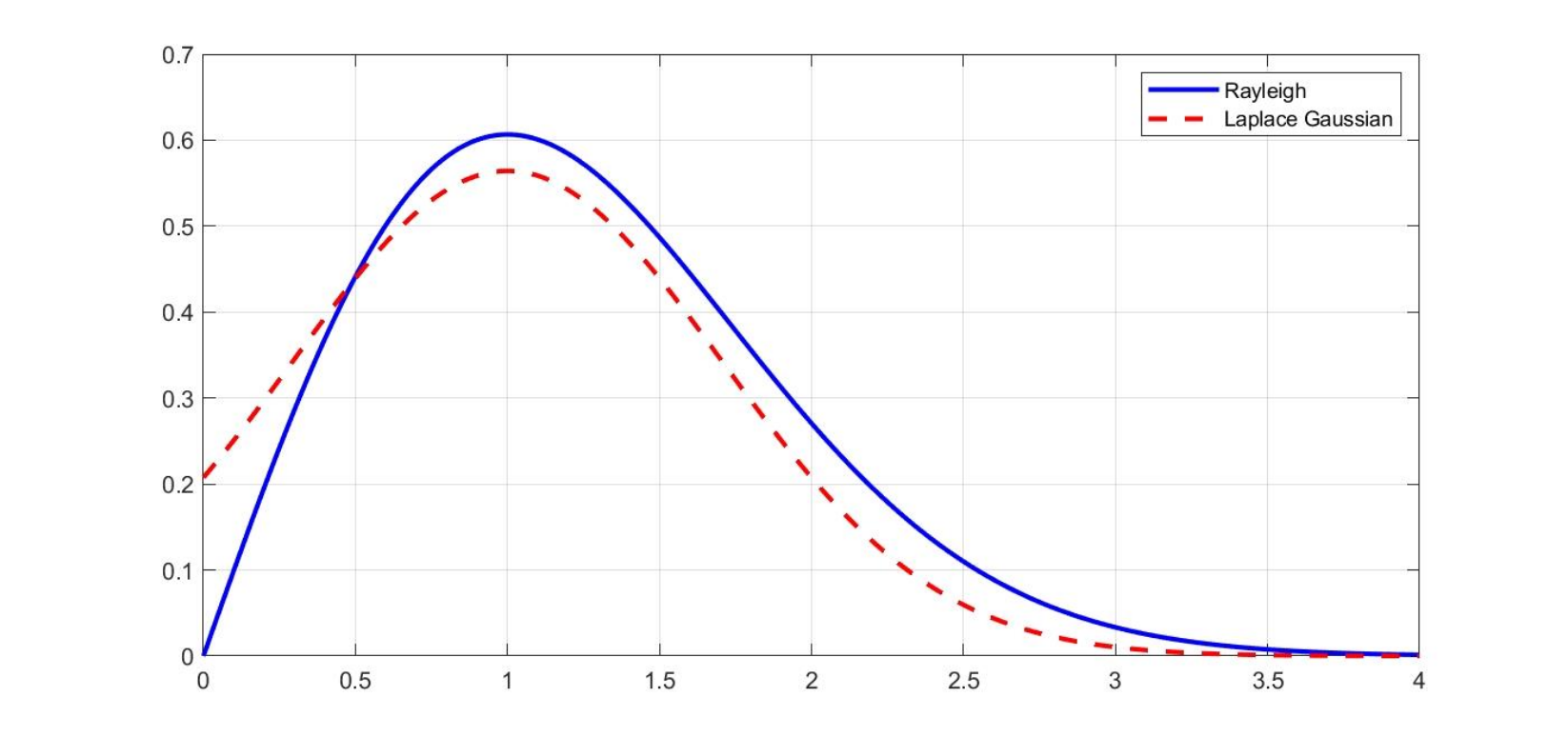
$$\sigma^2 = -\frac{1}{g''(\hat{x}^*)}$$

Example

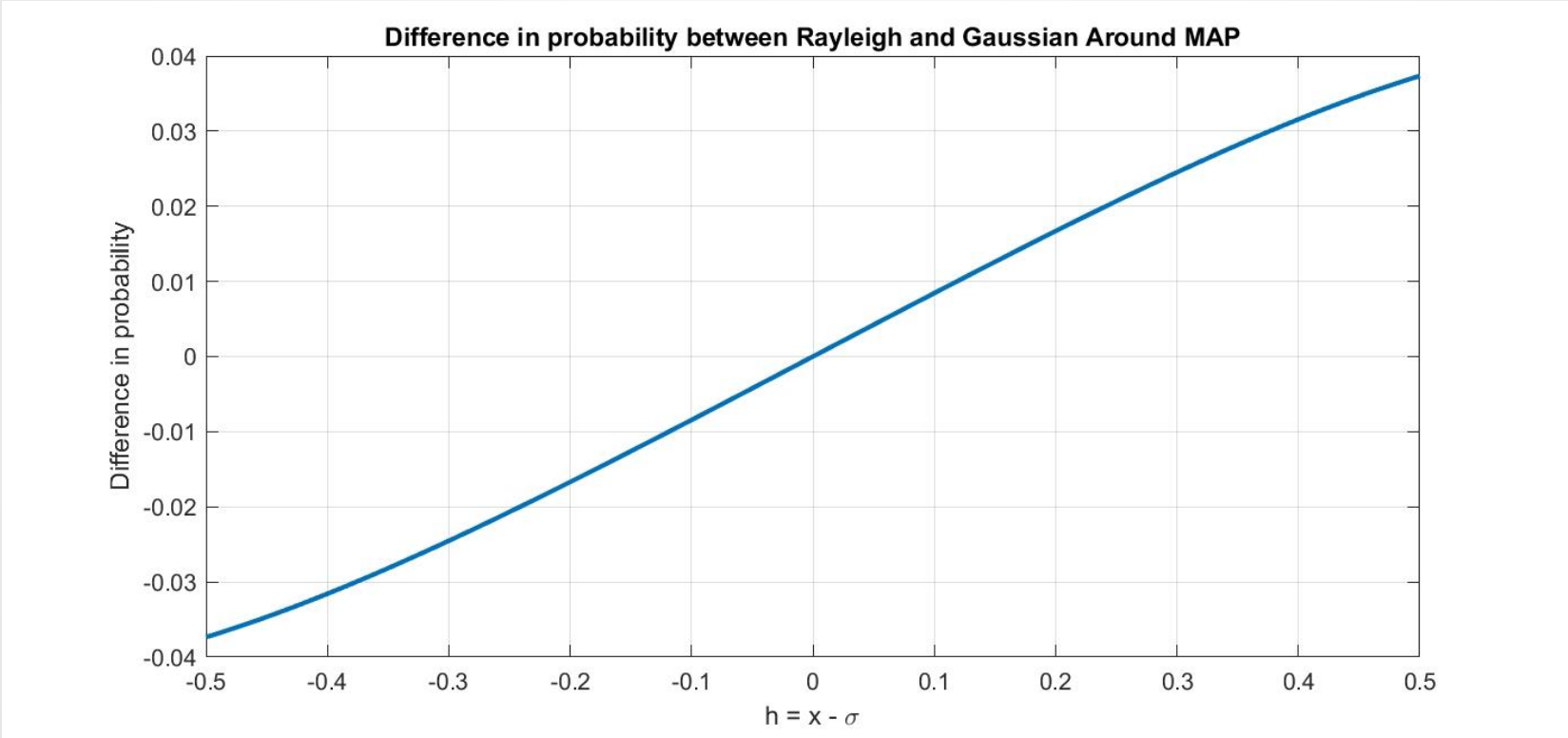


Log Rayleigh and Hessian of Log Rayleigh




0.7
0.6
0.5
0.4
0.3
0.2
0.1
0
0
0.5
1
1.5
2
2.5
3
3.5
4
Rayleigh
Laplace Gaussian

Difference in Probability



Multi-Dimensional Case

- In the multi-dimensional case, the covariance is given by the negative inverse of the Hessian,

$$\mathbf{P} = -\mathbf{H}^{-1}$$

Hessian

$$\mathbf{H} = \begin{bmatrix} \frac{\partial^2 g}{\partial x_1^2} & \frac{\partial^2 g}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 g}{\partial x_1 \partial x_n} \\ \frac{\partial^2 g}{\partial x_2 \partial x_1} & \frac{\partial^2 g}{\partial x_2^2} & \cdots & \frac{\partial^2 g}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 g}{\partial x_n \partial x_1} & \frac{\partial^2 g}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 g}{\partial x_2^2} \end{bmatrix}$$

Extracting Covariance in Practice

- To compute the covariance, naïvely we would have to invert $\mathbf{H}$
- However, in practice we just want the covariance of a small subset (e.g., blocks on the diagonals)
- Highly efficient approximate iterative schemes, such as the Takahashi inverse are used
- However, most of the time people avoid computing the covariance if they can get away with it

Common Optimization Algorithms

- *The Problem with Expected Values*
- *Maximum A Posteriori Estimation*
- *Optimization Algorithms*
- *Approximate Covariance Calculations*
- **Common Optimization Algorithms**

Newton-Raphson

- Gradient descent can find the local minimum
- However, its convergence can be very slow
- There are lots of variants of nonlinear optimization algorithms which have been derived
- Here we'll look at Newton, Gauss-Newton and Levenberg-Marquardt

Newton's Method

- The idea is to approximate the function by a quadratric and pick the step to optimize it
- The quadratic approximation is obtained from Taylor's series,

$$f(x+\delta x) \approx f(x) + f'(x)\delta x + \frac{1}{2} f''(x)\delta x^2$$

Newton's Method

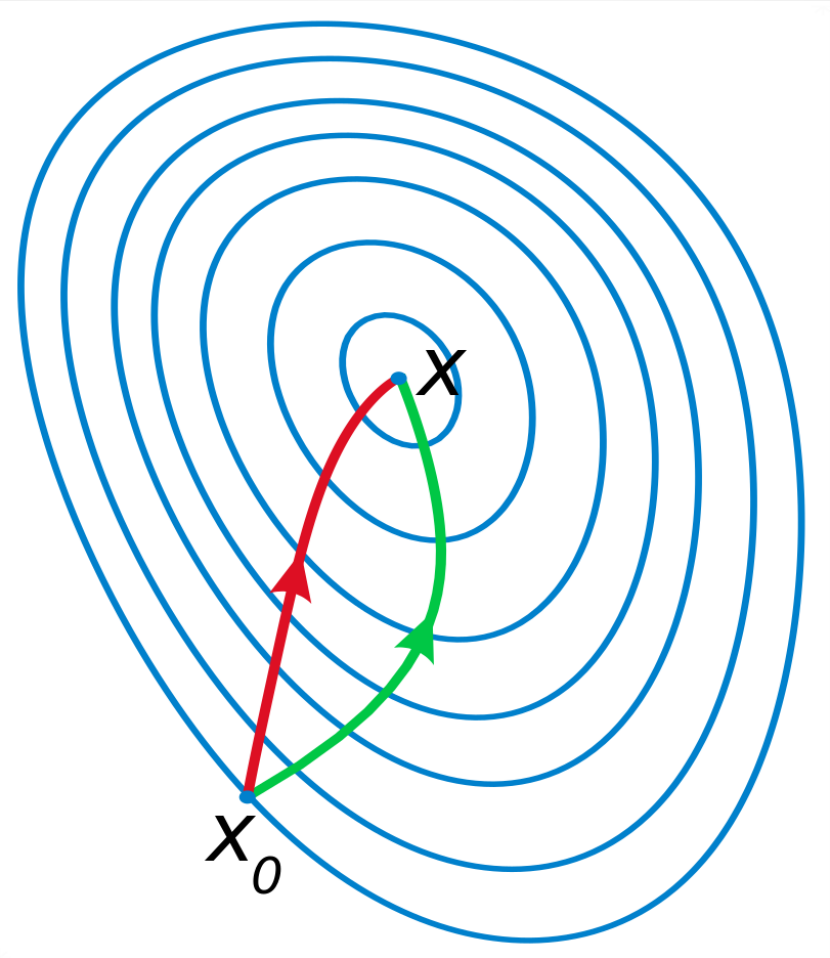
- **Taking derivatives with respect to the step size,**

$$\frac{\mathrm{d}f(x+\delta x)}{\mathrm{d}\delta x} = f'(x) + f''(x)\delta x$$

- **Since the left hand side is 0,**

$$\delta x = -\frac{f'(x)}{f''(x)}$$

Trajectory Taken by Newton's Method



From Newton's method in optimization

Newton's Method

- **The multi-dimensional version is**

$$\mathbf{X}_k^{i+1} = \mathbf{X}_k^i - \gamma \left[\mathbf{H}c\left(\mathbf{X}_k^i\right)\right]^{-1} \boldsymbol{\nabla} c\left(\mathbf{X}_k^i\right)$$

where

$$\mathbf{H}c\left(\mathbf{X}_k^i\right) = \begin{bmatrix} \frac{\partial^2 c\left(\mathbf{X}_k^i\right)}{\partial x_1^2} & \cdots & \frac{\partial^2 c\left(\mathbf{X}_k^i\right)}{\partial x_1 x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 c\left(\mathbf{X}_k^i\right)}{\partial x_n x_1} & \cdots & \frac{\partial^2 c\left(\mathbf{X}_k^i\right)}{\partial x_n^2} \end{bmatrix}^\top$$

Issues with Newton's Method

- It requires the calculation of the Hessian which can be *very expensive* to compute

- The method blows up if the Hessian is non positive definite

Non-Positiveness with the Hessian

- The easiest way to see this is with the scalar case
- We computed the turning point value

$$\delta x = -\frac{f'(x)}{f''(x)}$$

- However, the type of turning point depends on the second derivative

$$f''(x)$$

- If this is negative we have found a local maximum!

Gauss-Newton

- This uses an approximate value for the Hessian,

$$\mathbf{X}_k^{i+1} = \mathbf{X}_k^i - \gamma \left(\boldsymbol{\nabla} c \left(\mathbf{X}_k^i \right)^\top \boldsymbol{\nabla} c \left(\mathbf{X}_k^i \right) \right)^{-1} \boldsymbol{\nabla} c \left(\mathbf{X}_k^i \right)$$

- The approximate value is the “square” of the Jacobian
- You don’t have to compute second derivatives
- This is guaranteed to be positive semidefininte

Limitations with Gauss-Newton

- Gauss Newton is not guaranteed to find a local minimum
- We also have the problem that if the Jacobian is nearly singular, the step might be very inaccurately calculated, if it can be calculated at all
- Levenberg-Martquardt attempts to combine the “slow and safe” approach from Gradient Descent with ”faster and less safe” ness from Gauss Newton

Levenberg-Marquardt

- **The method introduces a damping factor:**

$$\mathbf{X}_k^{i+1} = \mathbf{X}_k^i - \gamma \left(\boldsymbol{\nabla} c \left(\mathbf{X}_k^i \right)^\top \boldsymbol{\nabla} c \left(\mathbf{X}_k^i \right) + \lambda \mathbf{I} \right)^{-1} \boldsymbol{\nabla} c \left(\mathbf{X}_k^i \right)$$

- **The damping factor is chosen dynamically, like the step size, to give the best result**

Levenberg-Marquardt Pseudo-Code

$\lambda \leftarrow 10^{-3}$;
Repeat
1. Compute the gradient vector $\mathbf{b}$ and the Hessian matrix $\mathbf{H}$;
2. Compute the increment $\widehat{\delta\theta} = -(\mathbf{H} + \lambda I)^{-1}\mathbf{b}$;
3. If $f(\theta + \widehat{\delta\theta}) < f(\theta)$
   - $\lambda \leftarrow \frac{\lambda}{10}$;

   Else
   - Repeat
     - $\lambda \leftarrow \lambda \times 10$;
     - Compute the increment $\widehat{\delta\theta} = -(\mathbf{H} + \lambda I)^{-1}\mathbf{b}$;

     Until $f(\theta + \widehat{\delta\theta}) < f(\theta)$;

   Endif
4. $\theta \leftarrow \theta + \widehat{\delta\theta}$;

Until convergence.

From [“HESSIAN MATRIX VS. GAUSS—NEWTON HESSIAN MATRIX”, P. Chen, 2011]

The Laplace Approximation in a Graph

- For the graph, it turns out the curvature is actually given by the inverse of the Hessian
- Using various approximations, its value is given by

$$\left(\boldsymbol{\nabla} c\left(\mathbf{X}_k^*\right)^\top \boldsymbol{\nabla} c\left(\mathbf{X}_k^*\right)\right)^{-1}$$

- This value was actually worked out as part of the Levenberg-Marquardt update step

Summary

- To extract a point estimate from the graph, we use maximum aposterior (MAP) estimates
- These do not require calculation of the normalization constant and tend to put the estimate in the correct place
- Computing the MAP estimate is a weighted nonlinear least squares estimation problem
- Recursive solutions exist to compute it